

Chapter 7

Graphs

Graph and its representations

-

A **Graph** is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(**V**) and a set of edges(**E**). The graph is denoted by **G(V, E)**.

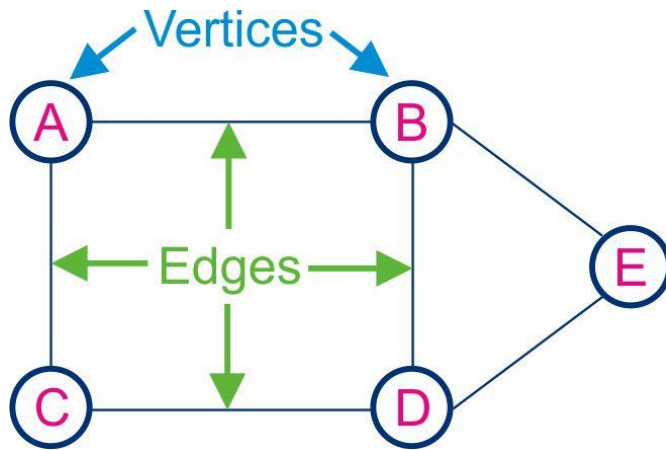
A real-life example of a graph in social media is the network of connections between users on a platform like Facebook, where each person is represented as a node and the "friend" relationship between them is an edge, effectively visualizing who is connected to whom within the social network.

This concept is used in many areas, like planning the best route on a road trip (where cities are nodes and roads are edges), understanding how diseases spread in a community (where people are nodes and their interactions are edges), or even figuring out how web pages are linked to each other on the internet.

A graph, in the context of mathematics and computer science, is a data structure used to represent relationships between entities. It consists of two sets:

Vertices (V): Represent the entities themselves, also called nodes or points.

Edges (E): Represent the relationships between the entities, also called links or lines.



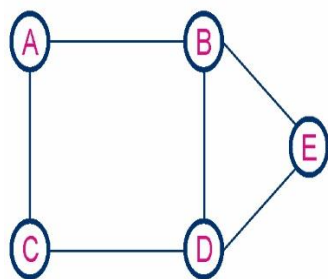
Definition

A graph is denoted as $G = (V, E)$, where:

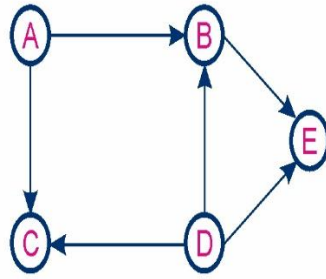
- V is a set of vertices.
- E is a set of edges.

These edges can be directed or undirected:

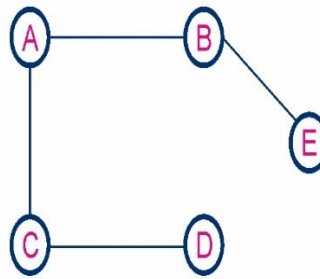
- **Directed edges:** Have an arrow indicating direction, showing how one vertex connects to another (e.g., $\mathbf{A} \rightarrow \mathbf{B}$).
- **Undirected edges:** Have no direction, representing a bi-directional connection between two vertices (e.g., $\mathbf{A} - \mathbf{B}$).



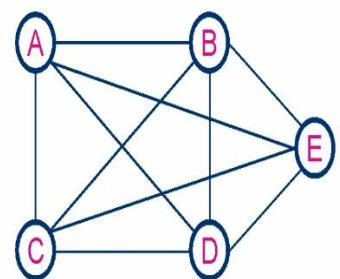
Undirected Graph



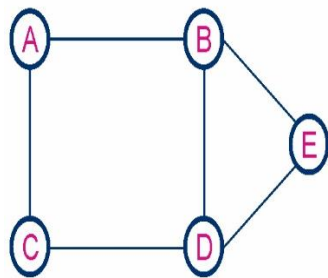
Directed Graph



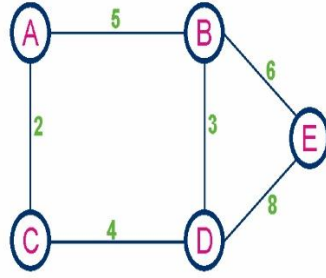
Sparse Graph



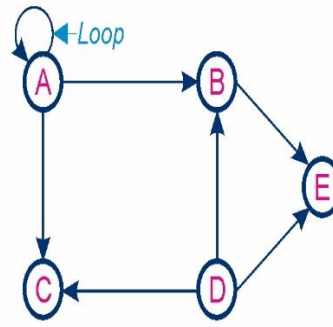
Complete Graph (Dense)



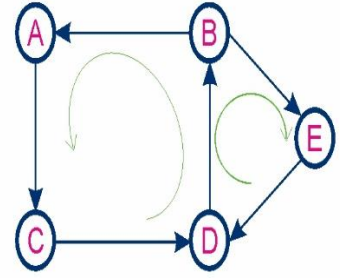
Unweighted Graph



Weighted Graph



Acyclic Graph



Cyclic Graph

Graph Terminology:

1. Graph

A Graph **G** is a non-empty set of vertices (or nodes) **V** and a set of edges **E**, where each edge connects a pair of vertices. Formally, a graph can be represented as **G= (V, E)**. Graphs can be classified based on various properties, such as directedness of edges and connectivity.

2. Vertex (Node)

A Vertex, often referred to as a **Node**, is a fundamental unit of a graph. It represents an **entity** within the graph. In applications like social networks, vertices can represent individuals, while in road networks, they can represent intersections or locations.

3. Edge

An Edge is a **connection between two vertices** in a graph. It can be either directed or undirected. In a directed graph, edges have a specific direction, indicating a one-way connection between vertices. In contrast, undirected

graphs have edges that do not have a direction and represent bidirectional connections.

4. Degree of a Vertex

The Degree of a Vertex in a graph is the **number of edges incident** to that vertex. In a directed graph, the degree is further categorized into the in-degree (number of incoming edges) and out-degree (number of outgoing edges) of the vertex.

5. Path

A Path in a graph is a **sequence of vertices** where each adjacent pair is connected by an edge. Paths can be of varying lengths and may or may not visit the same vertex more than once. The shortest path between two vertices is of particular interest in algorithms such as Dijkstra's algorithm for finding the shortest path in weighted graphs.

6. Cycle

A Cycle in a graph is a **path that starts and ends at the same vertex**, with no repetitions of vertices (except the starting and ending vertex, which are the same). Cycles are essential in understanding the connectivity and structure of a graph and play a significant role in cycle detection algorithms.
Advanced Graph Terminology:

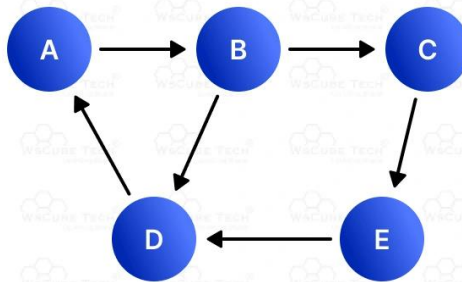
Types of Graph Data Structure With Examples

Graphs can be categorized based on their characteristics and properties:

1. Directed Graph

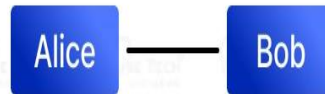
In a directed graph, edges have a direction, meaning they go from one node to another in a specific way.

Directed Graph



Example:

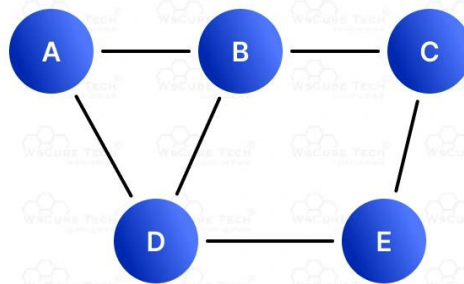
Think of a Twitter network where one person follows another. If Alice follows Bob, there is an edge from Alice to Bob but not necessarily from Bob to Alice.



2. Undirected Graph

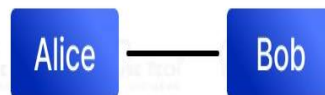
In an undirected graph, edges do not have a direction. They simply connect two nodes without any particular order.

Undirected Graph



Example:

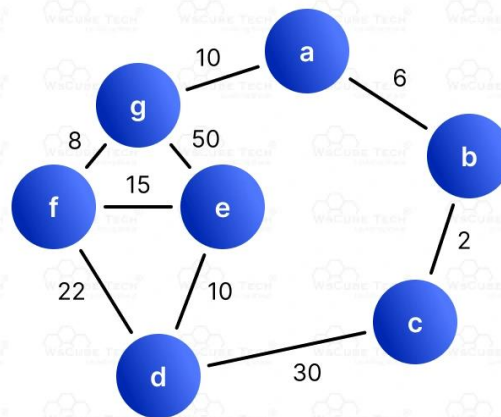
Think of a Facebook friendship where if Alice is friends with Bob, then Bob is also friends with Alice. The edge goes both ways.



3. Weighted Graph

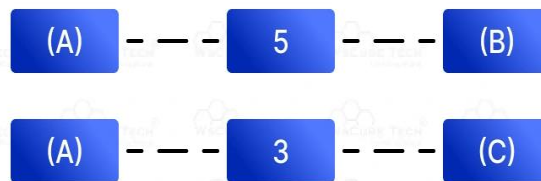
In a weighted graph, edges have weights or costs associated with them. These weights can represent distances, costs, or any other metric.

Weighted Graph



Example:

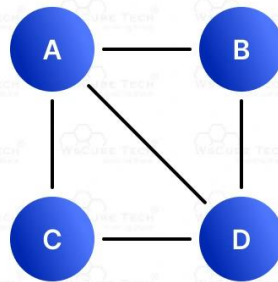
A road map where the weights on the edges represent the distance between cities.



4. Unweighted Graph

In an unweighted graph, all edges have the same weight, typically considered as 1.

Unweighted Graph



Example:

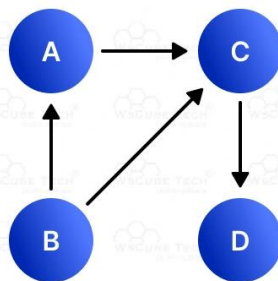
A simple social network where each friendship has the same importance.



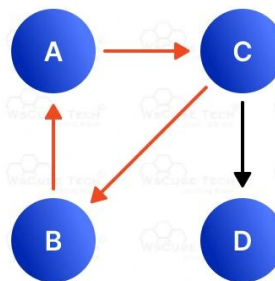
5. Cyclic Graph

A cyclic graph contains at least one cycle, meaning you can start at a node and follow a path that leads back to the same node

Acyclic

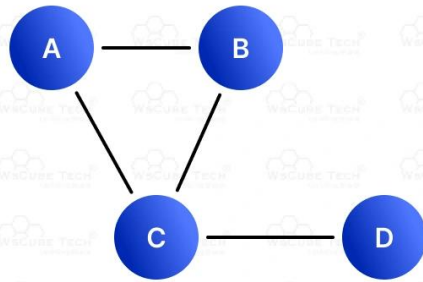


Cyclic



Example:

A graph representing routes between cities where some routes form a loop.



6. Acyclic Graph

An acyclic graph does not contain any cycles.

Example:

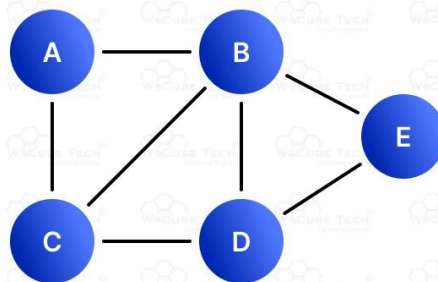
A family tree where no child node points back to its ancestors.



7. Connected Graph

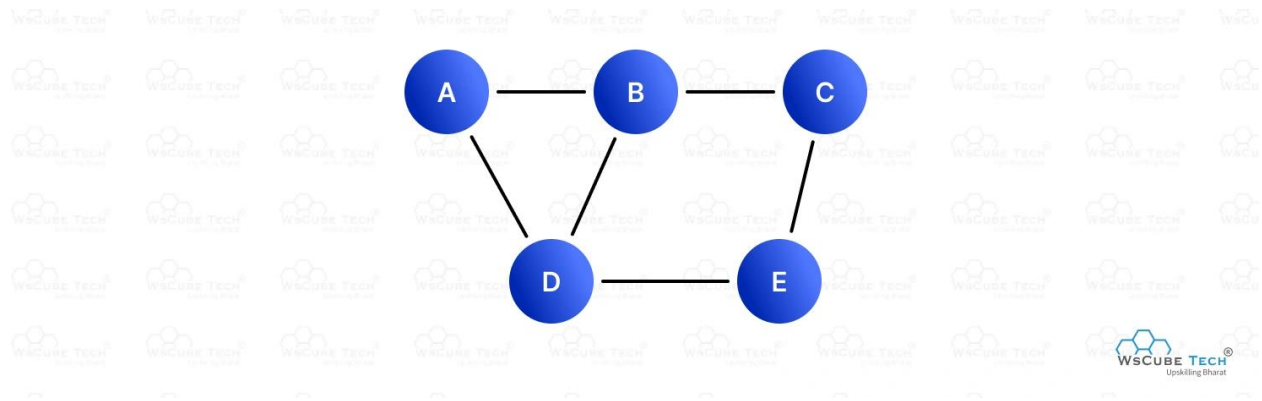
A connected graph has a path between every pair of nodes.

Connected Graph



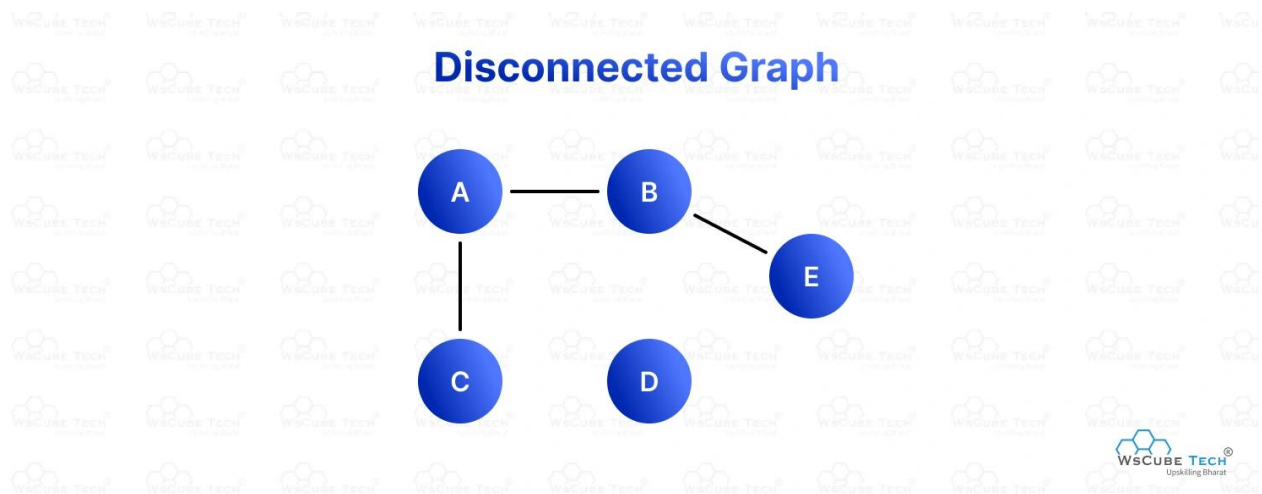
Example:

A small network where every computer can reach every other computer directly or indirectly.



8. Disconnected Graph

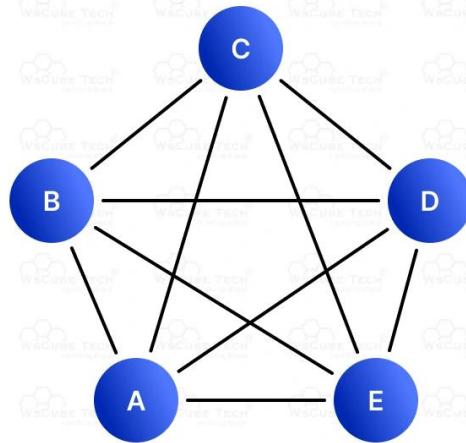
A disconnected graph has at least one pair of nodes with no path between them.



9. Complete Graph

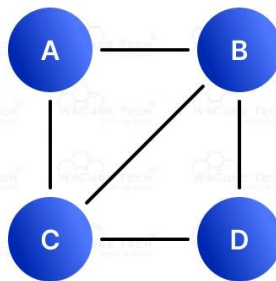
In a complete graph, there is an edge between every pair of nodes.

Complete Graph



Example:

A small social network where everyone is friends with everyone else.



Representation of Graphs:

Graphs are fundamental data structures used to represent relationships between entities. To store and manipulate these relationships efficiently, several different graph representations exist. Here's an overview of the most common ones:

1. Adjacency Matrix
2. Adjacency List

Adjacency Matrix

An adjacency matrix is a 2D array where each cell $[i, j]$ represents the presence or absence of an edge between vertices i and j . If the graph is weighted, the cell can store the weight of the edge.

Here's an overview of the key aspects of adjacency matrices:

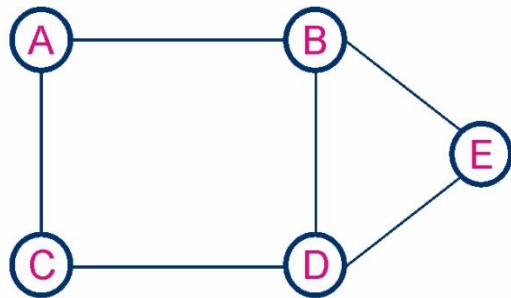
Adjacency Matrix Structure

Let $G=(V,E)$ be a graph with n nodes, $n>0$. a_{ij} is an adjacency matrix of G .

The a_{ij} is an $n \times n$ array whose elements are given by

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \in E \\ 0 & \text{otherwise} \end{cases}$$

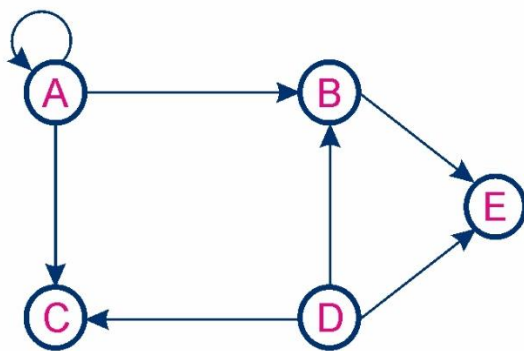
- The adjacency matrix is a square matrix of size $N \times N$, where N is the number of vertices in the graph.
- Each element (i, j) of the matrix represents the presence (1) or absence (0) of an edge between vertex i and vertex j .
- For **undirected graphs**, the matrix is symmetric, meaning $a[i][j] = a[j][i]$.
- For **directed graphs**, the adjacency matrix is not necessarily symmetric, meaning $a[i][j] \neq a[j][i]$.



Undirected Graph



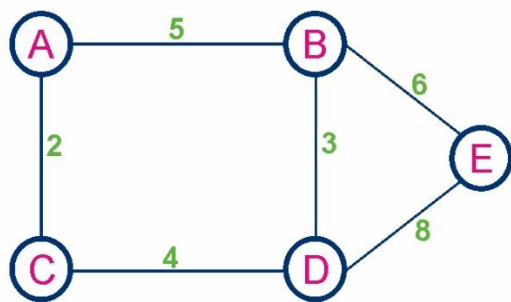
	A	B	C	D	E
A	0	1	1	0	0
B	1	0	0	1	1
C	1	0	0	1	0
D	0	1	1	0	1
E	0	1	0	1	0



Directed Graph



	A	B	C	D	E
A	1	1	1	0	0
B	0	0	0	0	1
C	0	0	0	0	0
D	0	1	1	0	1
E	0	0	0	0	0



Weighted Graph



	A	B	C	D	E
A	0	5	2	0	0
B	5	0	0	3	6
C	2	0	0	4	0
D	0	3	4	0	8
E	0	6	0	8	0

Advantages of Adjacency Matrix

- Efficient for dense graphs.
- Quick to check if there is an edge between two vertices.

Disadvantages of Adjacency Matrix

- Requires more space, as it allocates memory for every possible edge, which is impractical for large sparse graphs.
- Adding or removing vertices involves resizing the matrix, which can be inefficient.

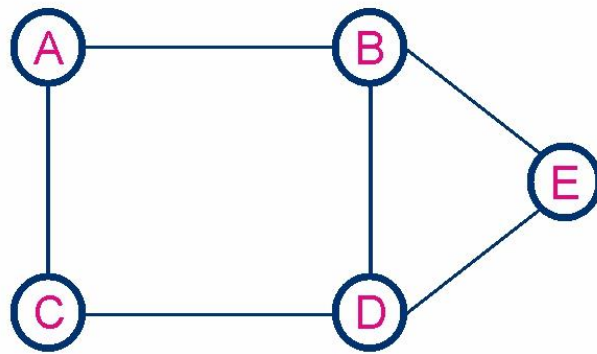
Adjacency List

The **n** rows of adjacency matrix are represented as **n linked lists**. There is one list for each vertex in the graph. The nodes in the list *i* represent the vertices that are adjacent from vertex *i*. Each node has at least two fields: **vertex and link**. The vertex field contains the indices of the vertices adjacent to vertex *i*. The adjacency lists for the undirected graph and directed graph, are illustrated below.

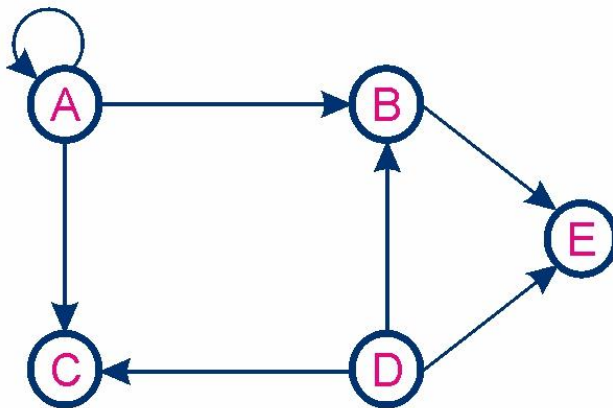
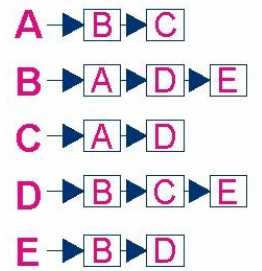
Here's an overview of the key features of adjacency lists:

Adjacency List Structure

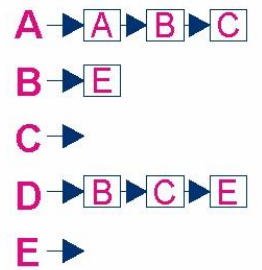
- Each vertex in the graph has a corresponding list.
- This list stores the identifiers of all the vertices connected to the original vertex by an edge.
- The list can be implemented using different data structures like arrays, linked lists, or even hash tables.
- The implementation choice depends on the desired functionality and performance characteristics.



Undirected Graph



Directed Graph



Advantages of Adjacency List

- Space-efficient for sparse graphs (graphs with relatively few edges compared to the number of vertices).
- Easy to add vertices and edges.
- Efficient in finding all adjacent vertices of a given vertex.

Disadvantages of Adjacency List

- Can be less efficient for dense graphs (where the number of edges is close to the maximum possible).

- Determining whether an edge exists between two vertices can be less efficient than with an adjacency matrix.

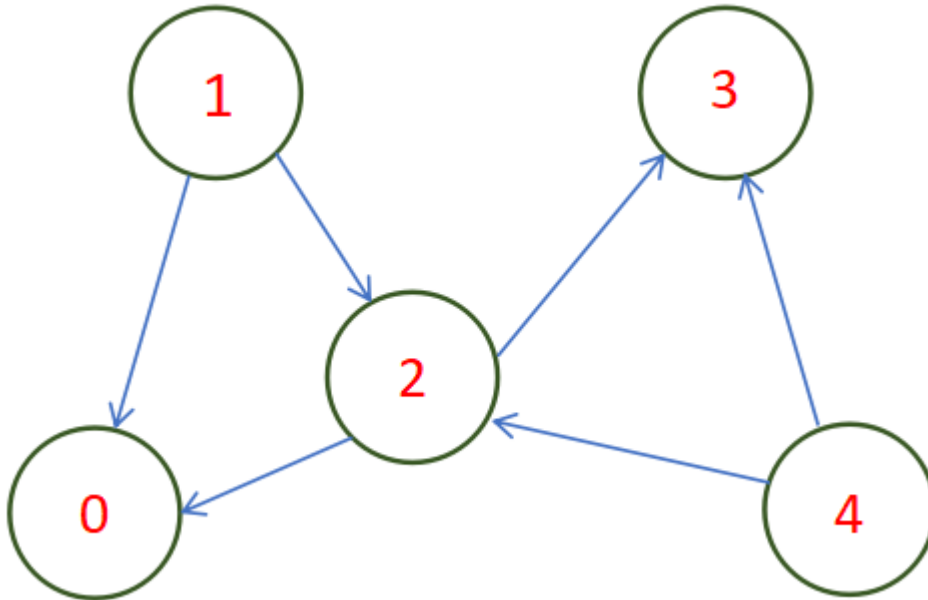
Transitive Closure

Definition:

The **transitive closure** of a graph is a way to determine which vertices are reachable from any other vertex in a directed graph. In simple terms, if there is a path from vertex u to vertex v , then v is reachable from u , and the transitive closure reflects this reachability in matrix form.

In any Directed Graph, let's consider a node i as a starting point and another node j as ending point. For all (i,j) pairs in a graph, transitive closure matrix is formed by the reachability factor, i.e if j is reachable from i (means there is a path from i to j) then we can put the matrix element as 1 or else if there is no path, then we can put it as 0.

Suppose we are given the following Directed Graph,



Then, the reachability matrix of the graph can be given by,

	0	1	2	3	4
0	1	0	0	0	0
1	1	1	1	1	0
2	1	0	1	1	0
3	0	0	0	0	0
4	1	0	1	1	1

This matrix is known as the transitive closure matrix, where '1' depicts the availability of a path from i to j , for each (i,j) in the matrix.

Warshall's Algorithm

Warshall's algorithm is a graph theory algorithm used to find the **transitive closure** of a directed graph. The transitive closure of a graph is a new graph that contains an edge between two vertices u and v if there is a path from u to v in the original graph.

Key Points:

1. **Input:** A directed graph represented by an adjacency matrix A , where:
 - $A[i][j]=1$ if there is a direct edge from vertex i to vertex j , and
 - $A[i][j]=0$ otherwise.
2. **Output:** The transitive closure matrix T , where:
 - $T[i][j]=1$ if there is a path (direct or indirect) from vertex i to vertex j , and

- $T[i][j]=0$ otherwise.

3. **Time Complexity:** $O(V^3)$, where V is the number of vertices in the graph.

4. **Algorithm Steps:**

- Use the adjacency matrix A as the initial matrix.
- Iterate through all vertices k as intermediate points.
- For each pair of vertices i and j , update:

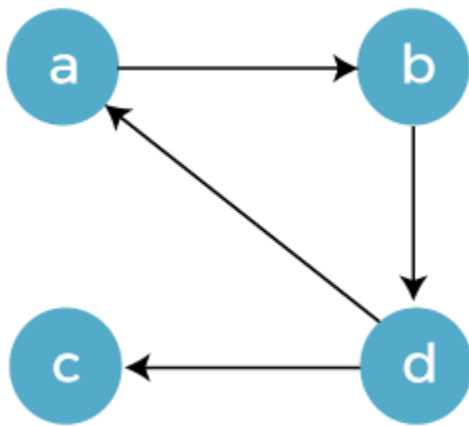
$$T[i][j] = T[i][j] \vee (T[i][k] \wedge T[k][j])$$

Where $\vee$ is the logical OR and $\wedge$ is the logical AND.

5. **Purpose:** Determines reachability between vertices in a graph, which is useful in various applications like network analysis, database query optimization, and state machine analysis.

Application of Warshall's algorithm to the directed graph

In order to understand this, we will use a graph, which is described as follow:



For this graph $R(0)$ will be looked like this:

$$R^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} \boxed{0} & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

Here $R(0)$ shows the adjacency matrix. In $R(0)$, we can see the existence of a path, which has no intermediate vertices. We will get $R(1)$ with the help of a boxed row and column in $R(0)$.

In $R(1)$, we can see the existence of a path, which has intermediate vertices. The number of the vertices is not greater than 1, which means just a vertex. It contains a new path from d to b. We will get $R(2)$ with the help of a boxed row and column in $R(1)$.

$$R^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \boxed{1} & 0 & 0 \\ \boxed{0} & \boxed{0} & \boxed{0} & \boxed{1} \\ 0 & 0 & 0 & 0 \\ 1 & \boxed{1} & 1 & 0 \end{bmatrix} \end{matrix}$$

In $R(2)$, we can see the existence of a path, which has intermediate vertices. The number of the vertices is not greater than 2, which means a and b. It contains two new paths. We will get $R(3)$ with the help of a boxed row and column in $R(2)$.

$$R^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & \mathbf{1} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & \mathbf{1} \end{bmatrix} \end{matrix}$$

In $R(3)$, we can see the existence of a path, which has intermediate vertices. The number of the vertices is not greater than 3, which means a, b and c. It does not contain any new paths. We will get $R(4)$ with the help of a boxed row and column in $R(3)$.

$$R^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

In $R(4)$, we can see the existence of a path, which has intermediate vertices. The number of the vertices is not greater than 4, which means a, b, c and d. It contains five new paths.

$$R^{(4)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} \mathbf{1} & 1 & \mathbf{1} & 1 \\ \mathbf{1} & \mathbf{1} & \mathbf{1} & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

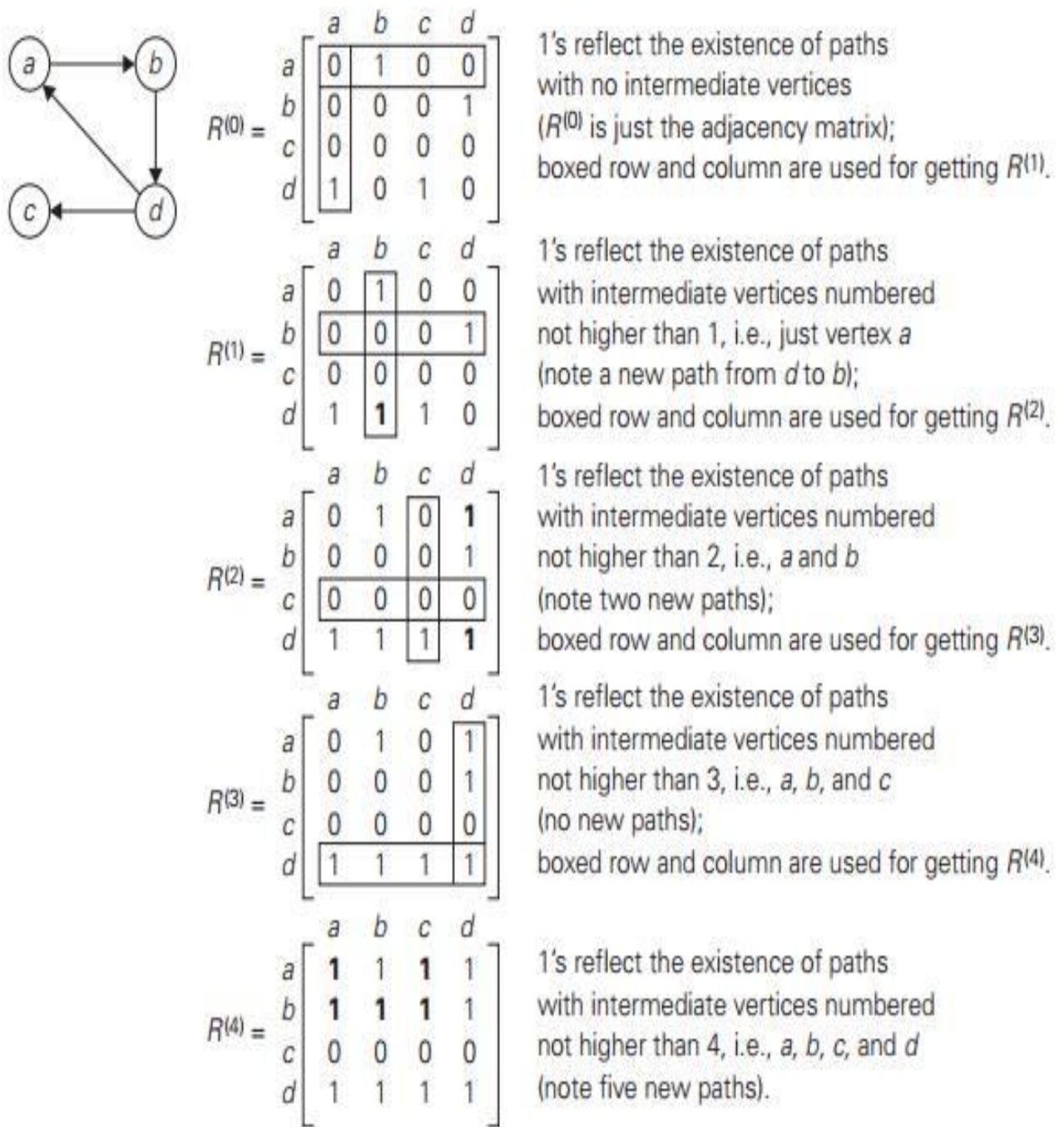


FIGURE 8.13 Application of Warshall's algorithm to the digraph shown. New 1's are in bold.

Graph traversals

Graph traversal means visiting every vertex and edge exactly once in a well-defined order. While using certain graph algorithms, you must ensure that each vertex of the graph is visited exactly once. The order in which the vertices are visited are important and may depend upon the algorithm or question that you are solving.

During a traversal, it is important that you track which vertices have been visited. The most common way of tracking vertices is to mark them.

Breadth First Search (BFS)

There are many ways to traverse graphs. BFS is the most commonly used approach.

BFS is a traversing algorithm where you should start traversing from a selected node (source or starting node) and traverse the graph layerwise thus exploring the neighbour nodes (nodes which are directly connected to source node). You must then move towards the next-level neighbour nodes.

BFS algorithm

A standard BFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

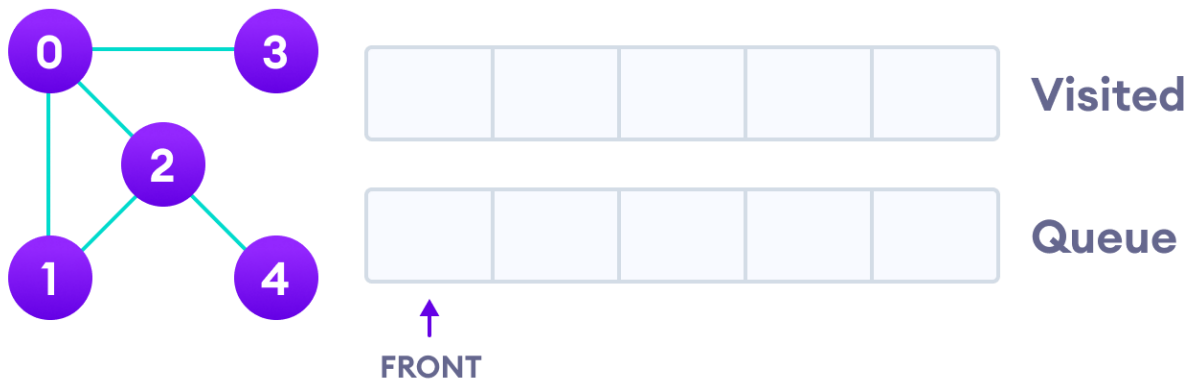
The algorithm works as follows:

1. Start by putting any one of the graph's vertices at the back of a queue.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

The graph might have two different disconnected parts so to make sure that we cover every vertex, we can also run the BFS algorithm on every node

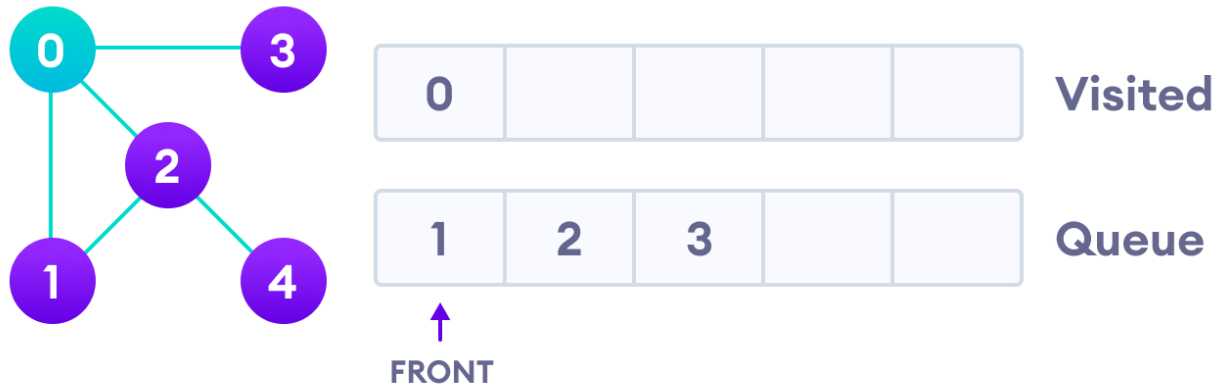
BFS example

Let's see how the Breadth First Search algorithm works with an example. We use an undirected graph with 5 vertices.



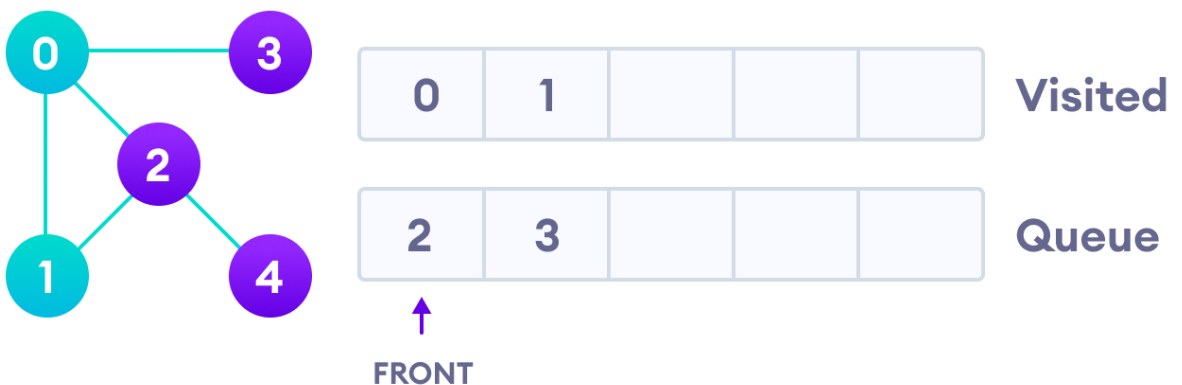
Undirected graph with 5 vertices

We start from vertex 0, the BFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the queue.



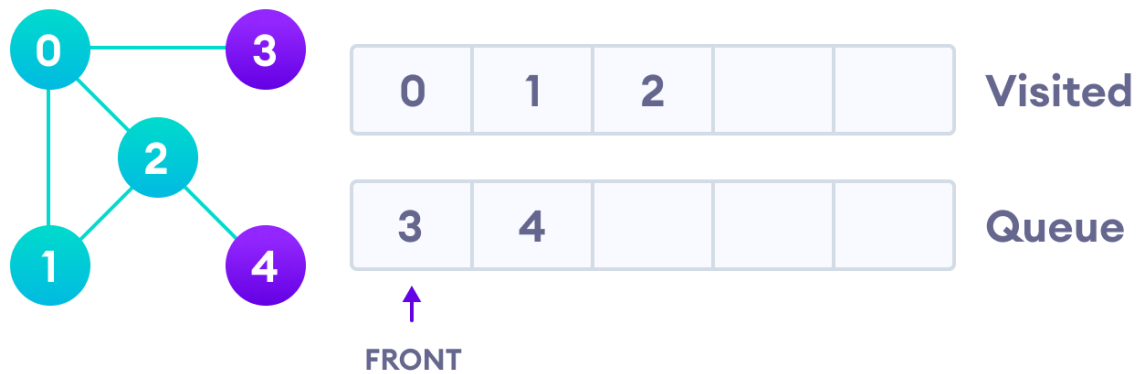
Visit start vertex and add its adjacent vertices to queue

Next, we visit the element at the front of queue i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

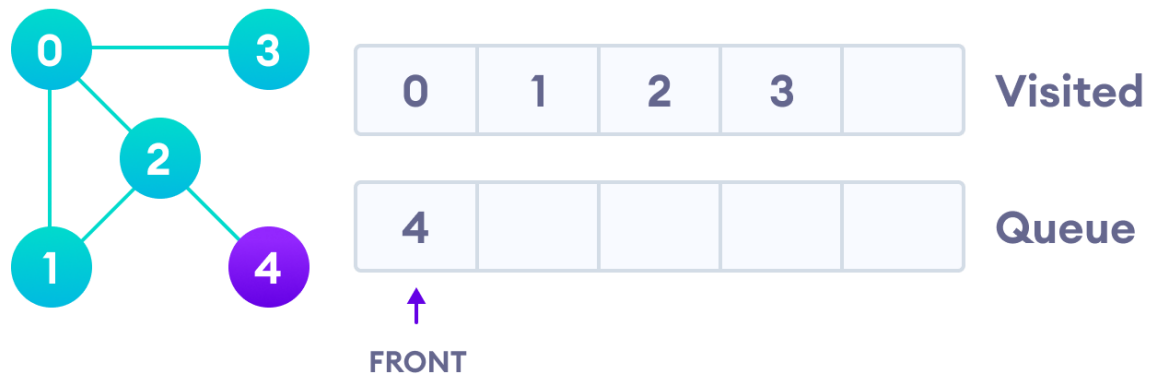


Visit the first neighbour of start node 0, which is 1

Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the back of the queue and visit 3, which is at the front of the queue.

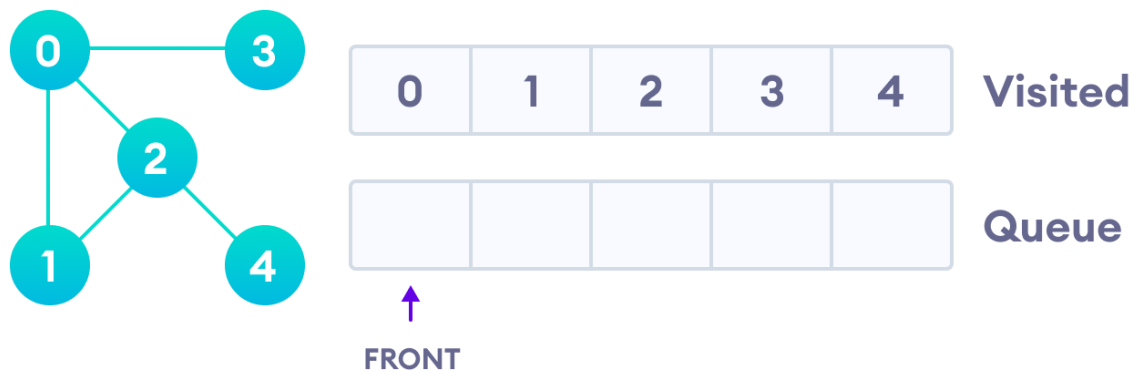


Visit 2 which was added to queue earlier to add its neighbours



4 remains in the queue

Only 4 remains in the queue since the only adjacent node of 3 i.e. 0 is already visited. We visit it.



Visit last remaining item in the queue to check if it has unvisited neighbors

Since the queue is empty, we have completed the Breadth First Traversal of the graph.

BFS Algorithm Complexity

The time complexity of the BFS algorithm is represented in the form of $O(V + E)$, where V is the number of nodes and E is the number of edges.

The space complexity of the algorithm is $O(V)$.

BFS Algorithm Applications

1. To build index by search index
2. For GPS navigation
3. Path finding algorithms
4. In Ford-Fulkerson algorithm to find maximum flow in a network

5. Cycle detection in an undirected graph
6. In [minimum spanning tree](#)

Depth First Traversal / DFS:

In Depth First Search (or DFS) for a graph, we traverse all adjacent vertices one by one. When we traverse an adjacent vertex, we completely finish the traversal of all vertices reachable through that adjacent vertex. This is similar to a tree, where we first completely traverse the left subtree and then move to the right subtree. The key difference is that, unlike trees, graphs may contain cycles (a node may be visited more than once). To avoid processing a node multiple times, we use a boolean visited array.

Depth First Search Algorithm

A standard DFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The DFS algorithm works as follows:

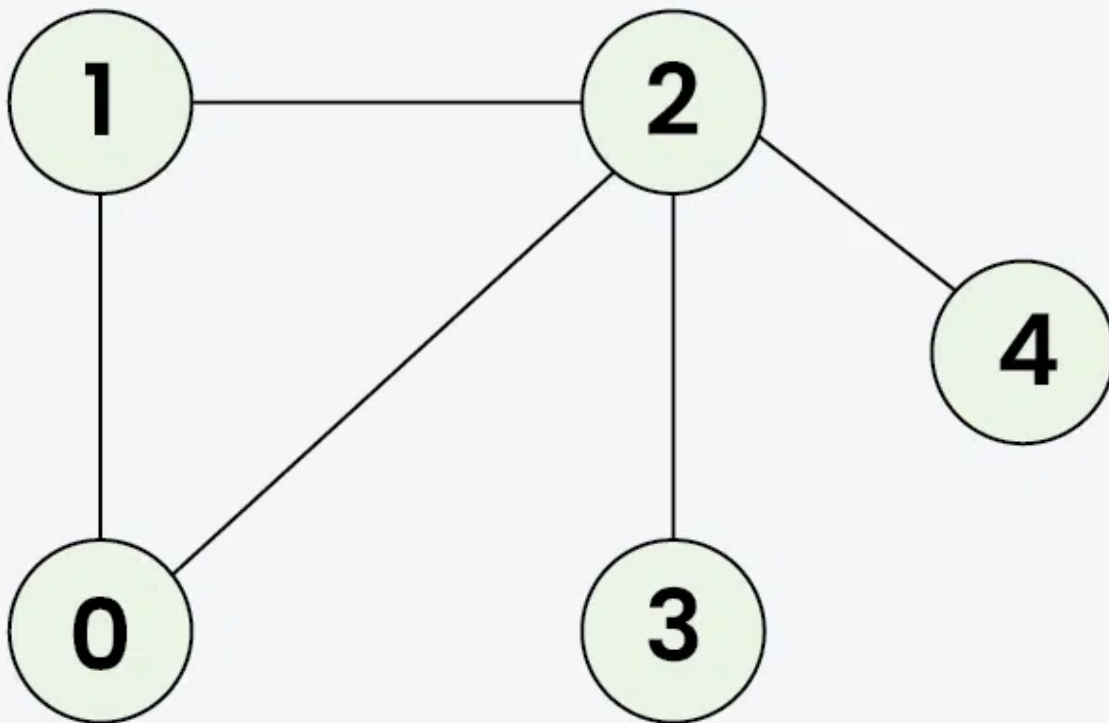
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.

3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.

Example:

Note : There can be multiple DFS traversals of a graph according to the order in which we pick adjacent vertices. Here we pick vertices as per the insertion order.

Input: $adj = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]$



Output: 1 2 0 3 4

Explanation: The source vertex s is 1. We visit it first, then we visit an adjacent.

Start at 1: Mark as visited. Output: 1

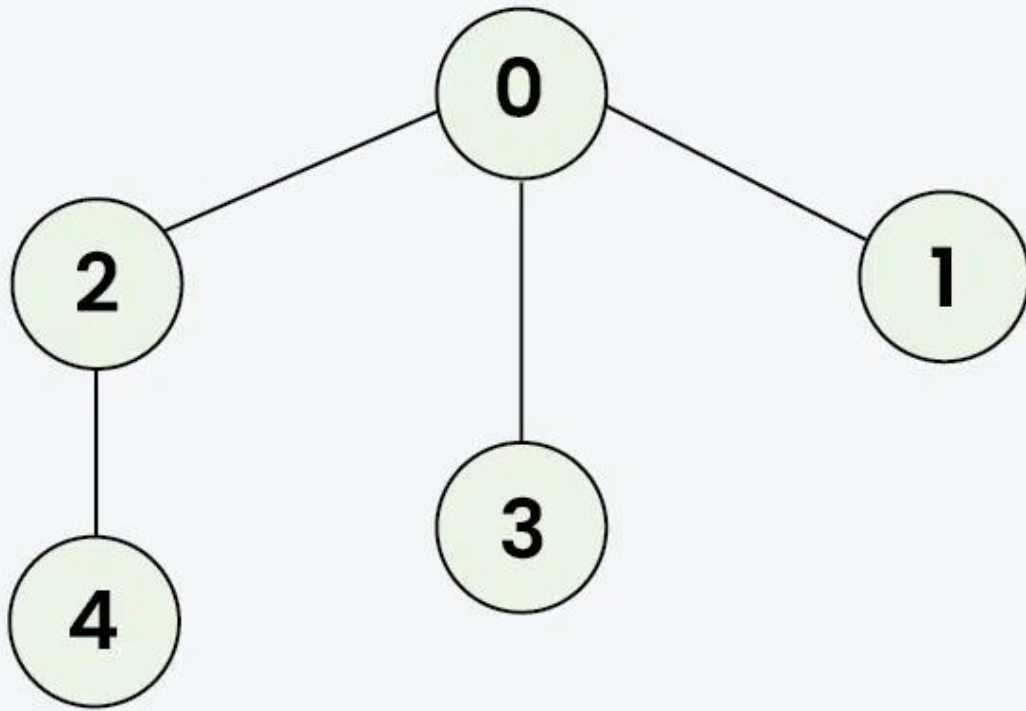
Move to 2: Mark as visited. Output: 2

Move to 0: Mark as visited. Output: 0 (backtrack to 2)

Move to 3: Mark as visited. Output: 3 (backtrack to 2)

Move to 4: Mark as visited. Output: 4 (backtrack to 1)

Q. **Input:** $[[2,3,1], [0], [0,4], [0], [2]]$



Output: 0 2 4 3 1

Explanation: DFS Steps:

Start at 0: Mark as visited. Output: 0

Move to 2: Mark as visited. Output: 2

Move to 4: Mark as visited. Output: 4 (backtrack to 2, then backtrack to 0)

Move to 3: Mark as visited. Output: 3 (backtrack to 0)

Move to 1: Mark as visited. Output: 1

Complexity of Depth First Search

The time complexity of the DFS algorithm is represented in the form of $O(V + E)$, where V is the number of nodes and E is the number of edges.

The space complexity of the algorithm is $O(V)$.

Application of DFS Algorithm

1. For finding the path
2. To test if the graph is bipartite
3. For finding the strongly connected components of a graph
4. For detecting cycles in a graph

What is Minimum Spanning Tree (MST)

-

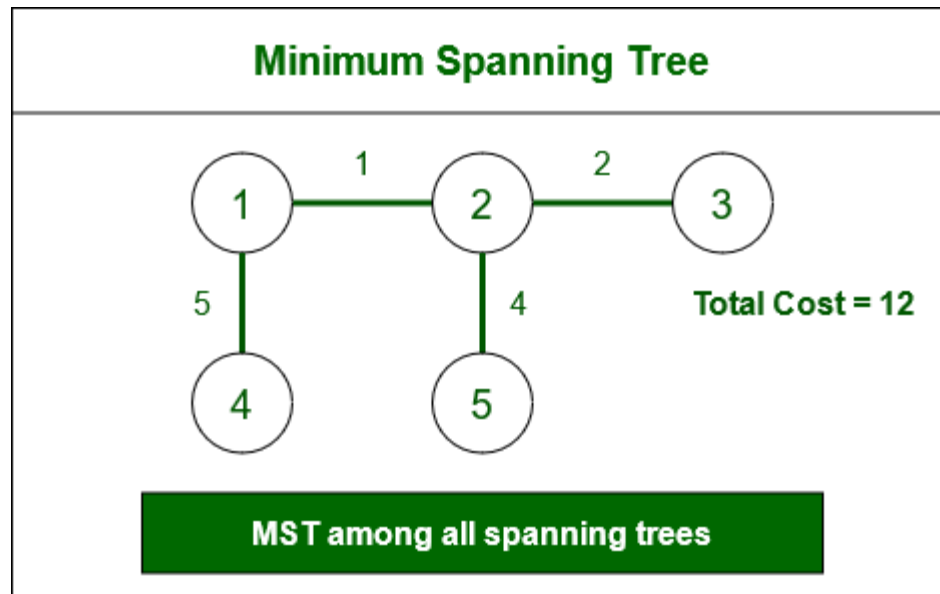
A **spanning tree** is defined as a tree-like subgraph of a connected, undirected graph that includes all the vertices of the graph. Or, to say in Layman's words, it is a subset of the edges of the graph that forms a tree (**acyclic**) where every node of the graph is a part of the tree.

The minimum spanning tree has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees. Like a spanning tree, there can also be many possible MSTs for a graph.

*A **minimum spanning tree (MST)** is defined as a **spanning tree** that has the minimum weight among all the possible spanning trees.*

The minimum spanning tree has all the properties of a spanning tree with an added constraint of having the minimum possible weights among all possible spanning trees. Like a spanning tree, there can also be many possible MSTs for a graph.

- Let's look at the MST of the above example Graph,



Algorithms to find Minimum Spanning Tree:

There are several algorithms to find the minimum spanning tree from a given graph, some of them are listed below:

Kruskal's Minimum Spanning Tree Algorithm:

This is one of the popular algorithms for finding the minimum spanning tree from a connected, undirected graph. This is a **greedy algorithm**. The algorithm workflow is as below:

- First, it sorts all the edges of the graph by their weights,
- Then starts the iterations of finding the spanning tree.
- At each iteration, the algorithm adds the next lowest-weight edge one by one, such that the edges picked until now does not form a cycle.

This algorithm can be implemented efficiently using a DSU (Disjoint-Set) data structure to keep track of the connected components of the graph. This is used in a variety of practical applications such as network design, clustering, and data analysis.

How Kruskal's algorithm works

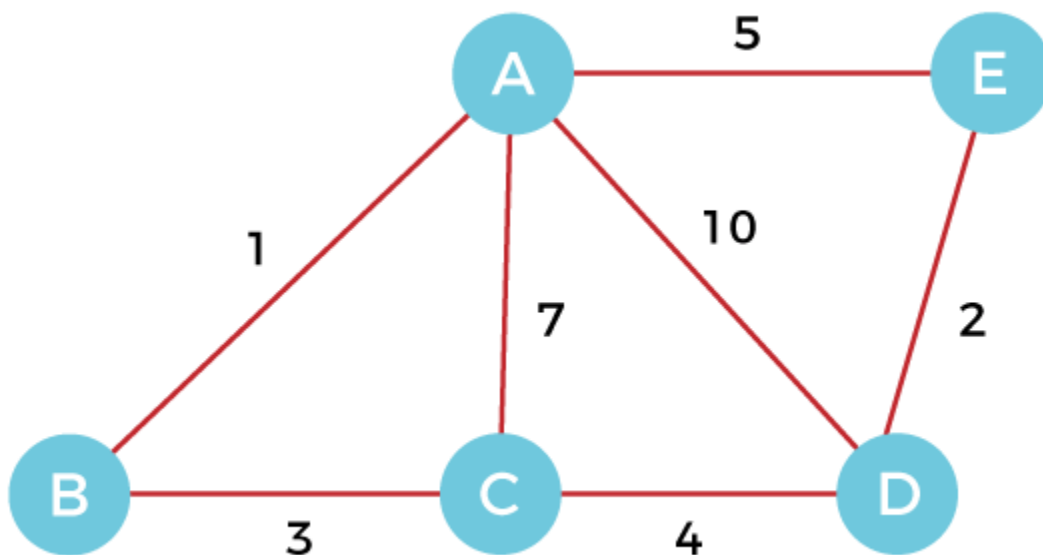
The steps for implementing Kruskal's algorithm are as follows:

1. Sort all the edges from low weight to high
2. Take the edge with the lowest weight and add it to the spanning tree. If adding the edge created a cycle, then reject this edge.
3. Keep adding edges until we reach all vertices.

Example of Kruskal's algorithm

Now, let's see the working of Kruskal's algorithm using an example. It will be easier to understand Kruskal's algorithm using an example.

Suppose a weighted graph is -



The weight of the edges of the above graph is given in the below table –

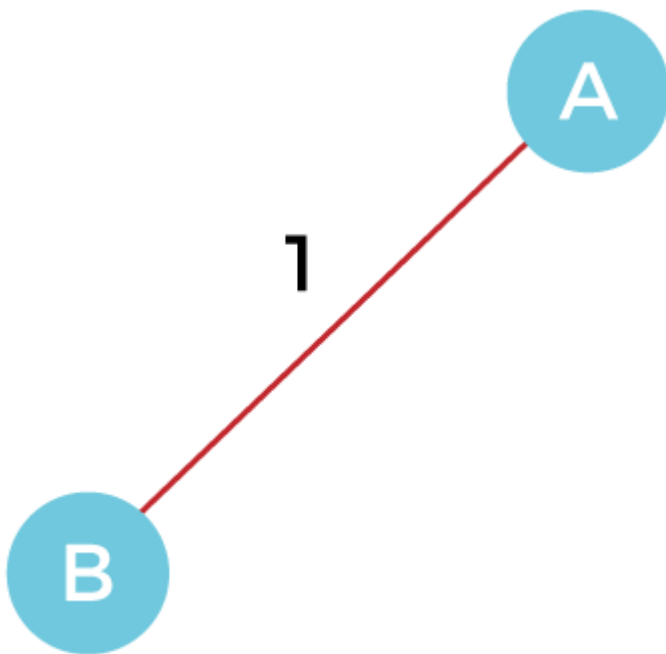
Edge	AB	AC	AD	AE	BC	CD	DE
Weight	1	7	10	5	3	4	2

Now, sort the edges given above in the ascending order of their weights.

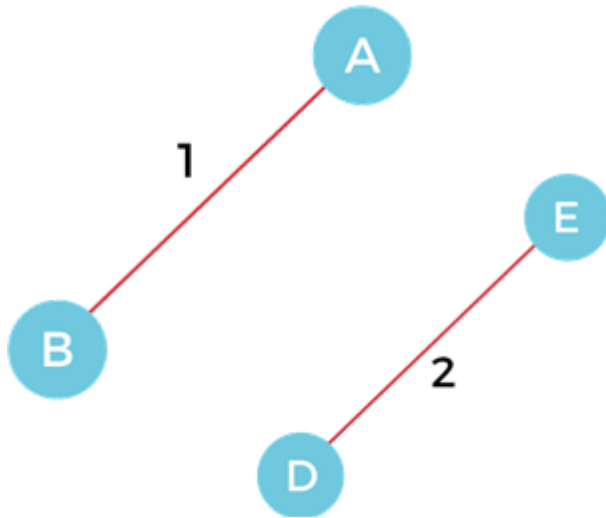
Edge	AB	DE	BC	CD	AE	AC	AD
Weight	1	2	3	4	5	7	10

Now, let's start constructing the minimum spanning tree.

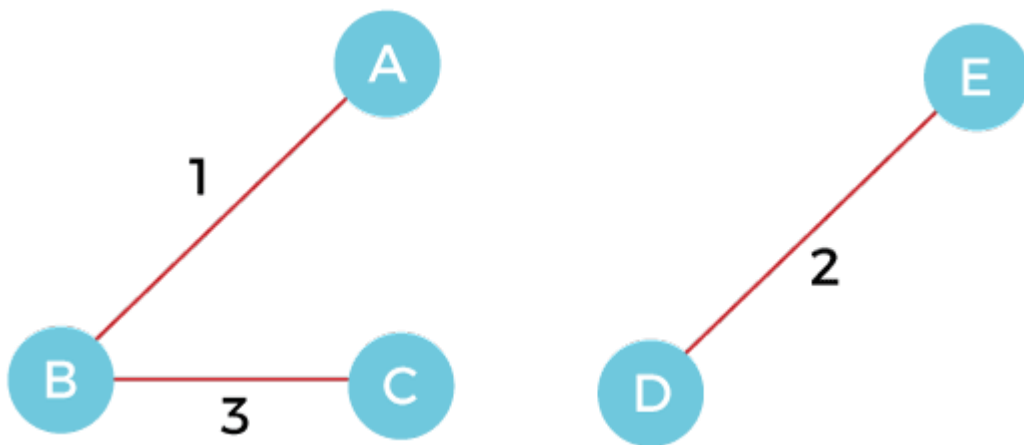
Step 1 - First, add the edge **AB** with weight **1** to the MST.



Step 2 - Add the edge **DE** with weight **2** to the MST as it is not creating the cycle.

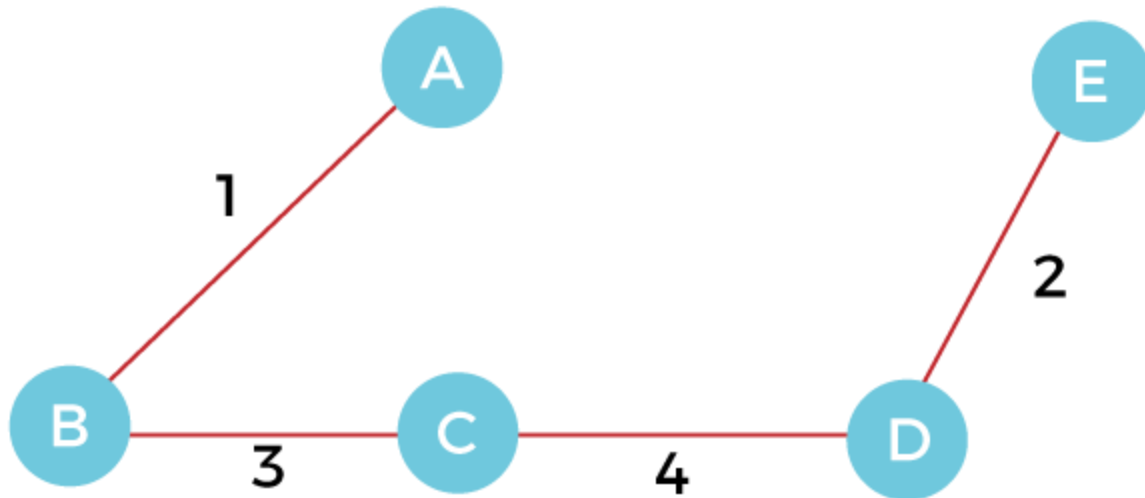


Step 3 - Add the edge **BC** with weight **3** to the MST, as it is not creating any cycle or loop.



Step 4 - Now, pick the edge **CD** with weight **4** to the MST, as it is not forming the cycle.

AD

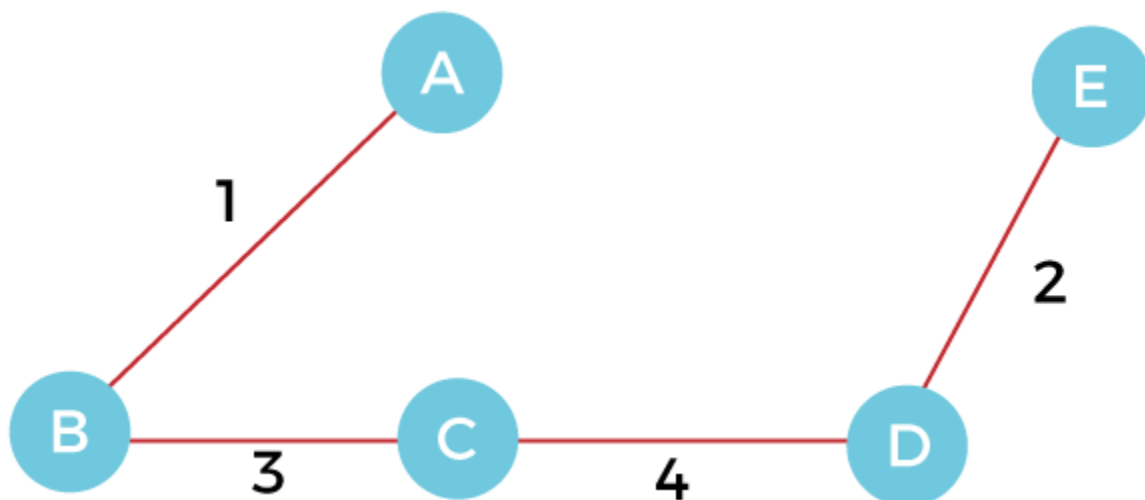


Step 5 - After that, pick the edge **AE** with weight **5**. Including this edge will create the cycle, so discard it.

Step 6 - Pick the edge **AC** with weight **7**. Including this edge will create the cycle, so discard it.

Step 7 - Pick the edge **AD** with weight **10**. Including this edge will also create the cycle, so discard it.

So, the final minimum spanning tree obtained from the given weighted graph by using Kruskal's algorithm is -



The cost of the MST is = $AB + DE + BC + CD = 1 + 2 + 3 + 4 = 10$.

Now, the number of edges in the above tree equals the number of vertices minus 1. So, the algorithm stops here.

$$E = V - 1$$

Complexity of Kruskal's algorithm

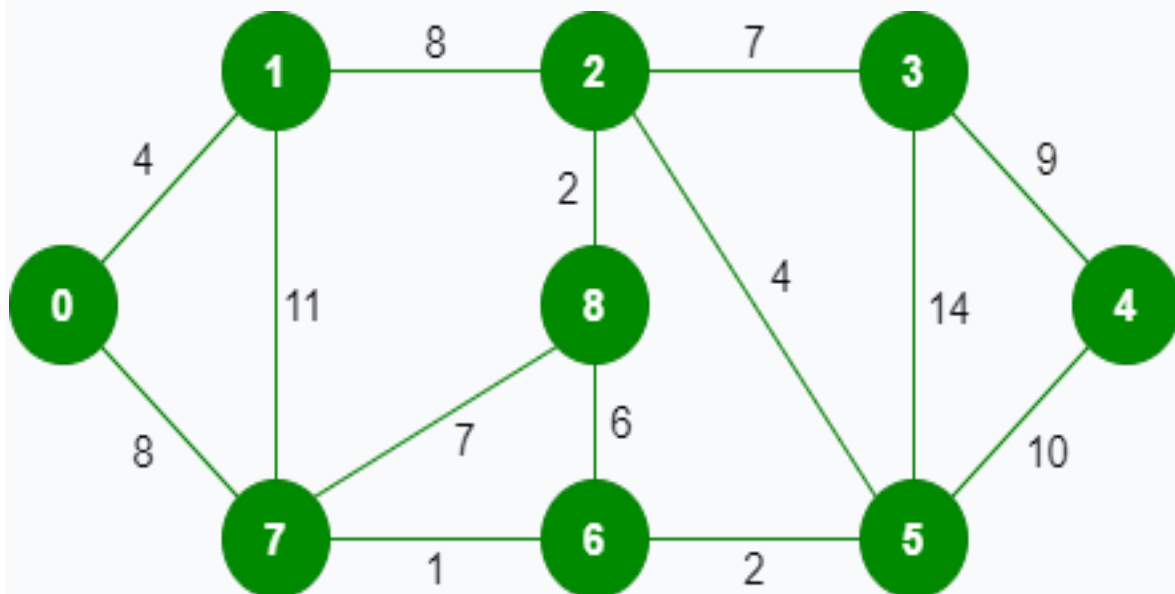
Now, let's see the time complexity of Kruskal's algorithm.

Time Complexity

The time complexity of Kruskal's algorithm is $O(E \log E)$ or $O(V \log V)$, where E is the no. of edges, and V is the no. of vertices.

Auxiliary Space: $O(V + E)$, where V is the number of vertices and E is the number of edges in the graph.

Q1. Solve this:



Prim's Algorithm:

Prim's Algorithm is a greedy algorithm that is used to find the minimum spanning tree from a graph. Prim's algorithm finds the subset of edges that includes every vertex of the graph such that the sum of the weights of the edges can be minimized.

How does the prim's algorithm work?

Prim's algorithm is a greedy algorithm that starts from one vertex and continue to add the edges with the smallest weight until the goal is reached. The steps to implement the prim's algorithm are given as follows -

- First, we have to initialize an MST with the randomly chosen vertex.
- Now, we have to find all the edges that connect the tree in the above step with the new vertices. From the edges found, select the minimum edge and add it to the tree.
- Repeat step 2 until the minimum spanning tree is formed.

The applications of prim's algorithm are -

- Prim's algorithm can be used in network designing.
- It can be used to make network cycles.
- It can also be used to lay down electrical wiring cables.

Prim's algorithm starts with the single node and explores all the adjacent nodes with all the connecting edges at every step. The edges with the minimal weights causing no cycles in the graph got selected.

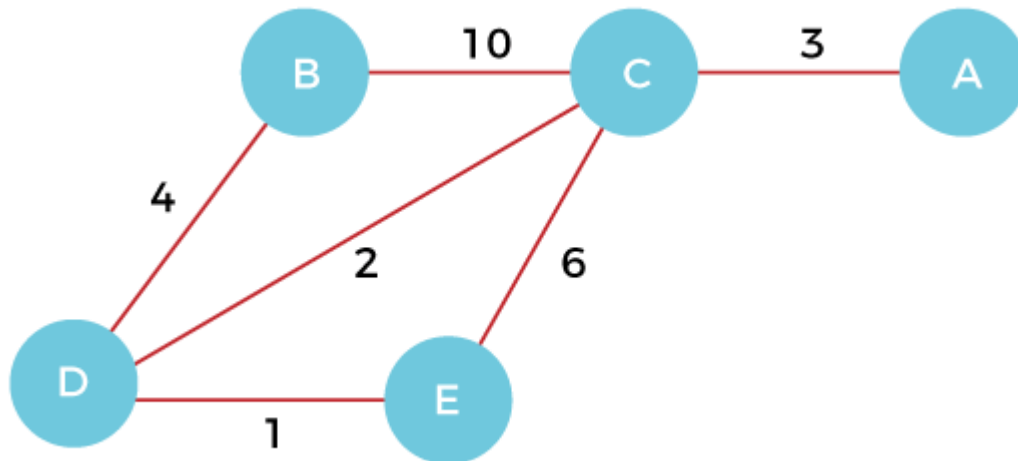
Algorithm

1. Step 1: Select a starting vertex
2. Step 2: Repeat Steps 3 and 4 until there are fringe vertices
3. Step 3: Select an edge 'e' connecting the tree vertex and fringe vertex that has minimum weight
4. Step 4: Add the selected edge and the vertex to the minimum spanning tree T
5. [END OF LOOP]
6. Step 5: EXIT

Example of prim's algorithm

Now, let's see the working of prim's algorithm using an example. It will be easier to understand the prim's algorithm using an example.

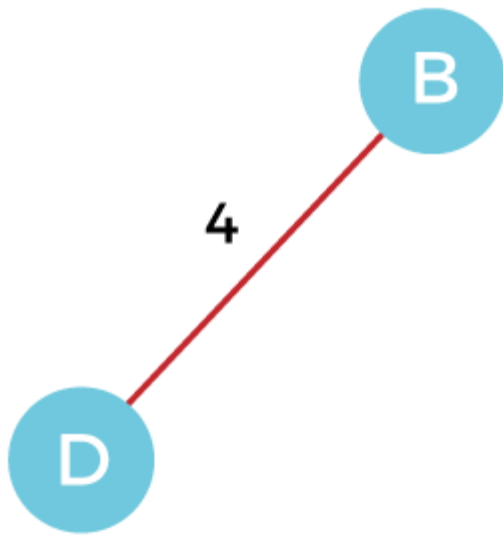
Suppose, a weighted graph is -



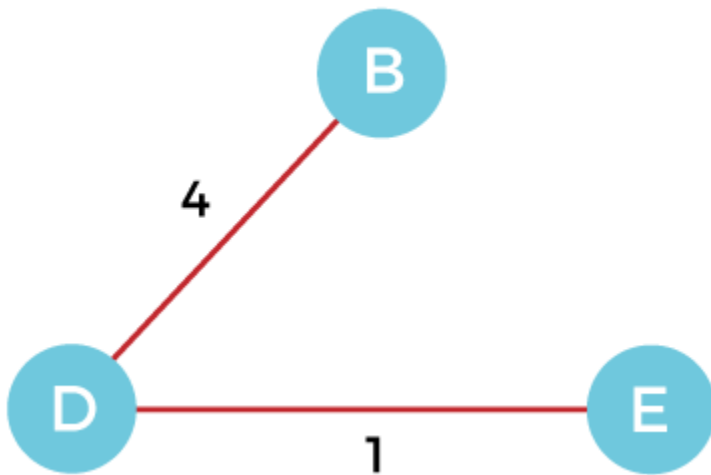
Step 1 - First, we have to choose a vertex from the above graph. Let's choose B.



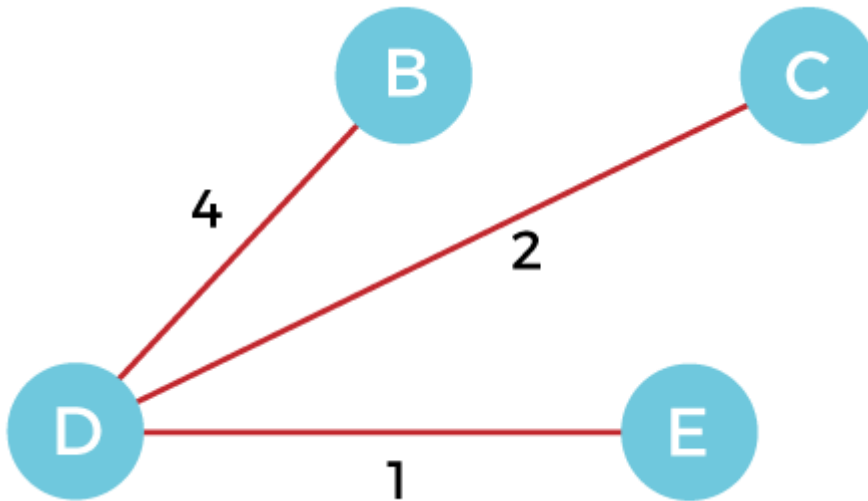
Step 2 - Now, we have to choose and add the shortest edge from vertex B. There are two edges from vertex B that are B to C with weight 10 and edge B to D with weight 4. Among the edges, the edge BD has the minimum weight. So, add it to the MST.



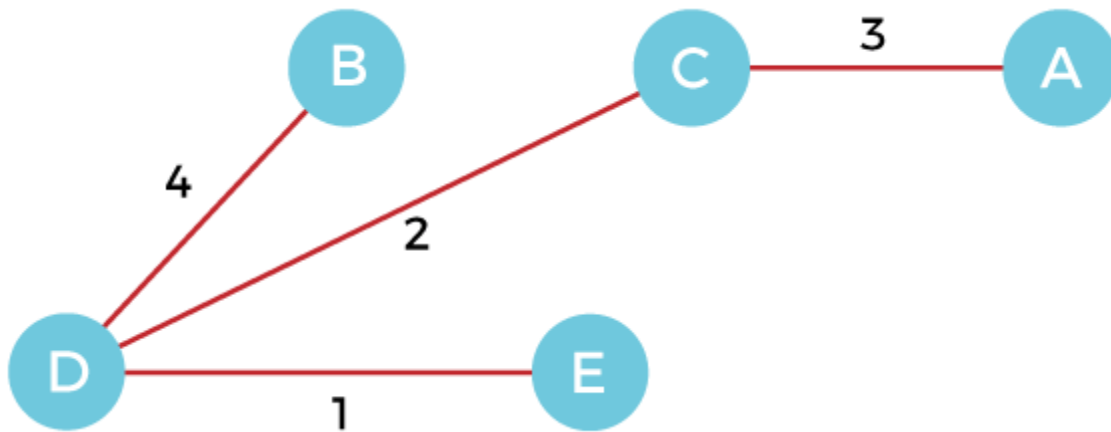
Step 3 - Now, again, choose the edge with the minimum weight among all the other edges. In this case, the edges DE and CD are such edges. Add them to MST and explore the adjacent of C, i.e., E and A. So, select the edge DE and add it to the MST.



Step 4 - Now, select the edge CD, and add it to the MST.



Step 5 - Now, choose the edge CA. Here, we cannot select the edge CE as it would create a cycle to the graph. So, choose the edge CA and add it to the MST.



So, the graph produced in step 5 is the minimum spanning tree of the given graph. The cost of the MST is given below -

Cost of MST = $4 + 2 + 1 + 3 = 10$ units.

Complexity Analysis of Prim's Algorithm:

Time Complexity: $O(V^2)$, If the input graph is represented using an adjacency list, then the time complexity of Prim's algorithm can be reduced to $O(E * \log V)$ with the help of a binary heap. In this implementation, we are always considering the spanning tree to start from the root of the graph.

Auxiliary Space: $O(V)$

Shortest Path Problems

Shortest path problems in **Data Structures and Algorithms (DSA)** involve finding the minimum distance or cost to travel between two vertices in a weighted graph. There are several algorithms to solve different types of shortest path problems:

Types of Shortest Path Problems

1. **Single Source Shortest Path (SSSP):** Find the shortest path from a single source vertex to all other vertices.
 - **Algorithms:** Dijkstra's Algorithm, Bellman-Ford Algorithm
 -
2. **All Pairs Shortest Path (APSP):** Find the shortest path between every pair of vertices.
 - **Algorithms:** Floyd-Warshall Algorithm, Johnson's Algorithm
 -
3. **Single Pair Shortest Path:** Find the shortest path between a specific pair of vertices.
 - **Algorithms:** Dijkstra's Algorithm (for positive weights), A* Algorithm (for heuristics)
 -
4. **K Shortest Paths:** Find the top K shortest paths between two vertices.
 - **Algorithms:** Yen's Algorithm

Dijkstra's Algorithm (Single-Shortest Path Algorithm):

Dijkstra's Algorithm. Dijkstra's Algorithm is a Graph algorithm that finds the shortest path from a source vertex to all other vertices in the Graph (single source shortest path). It is a type of Greedy Algorithm that only works on Weighted Graphs having positive weights. The time complexity of Dijkstra's Algorithm is $O(V^2)$ with the help of the adjacency

matrix representation of the graph. This time complexity can be reduced to **$O((V + E) \log V)$** with the help of an adjacency list representation of the graph, where **V** is the number of vertices and **E** is the number of edges in the graph.

Relaxation:

If $d(u) + C(u,v) < d(v)$

$$d(v) = d(u) + C(u,v)$$

Algorithm for Dijkstra's Algorithm:

1. Mark the source node with a current distance of 0 and the rest with infinity.
2. Set the non-visited node with the smallest current distance as the current node.
3. For each neighbor, N of the current node adds the current distance of the adjacent node with the weight of the edge connecting 0->1. If it is smaller than the current distance of Node, set it as the new current distance of N.
4. Mark the current node 1 as visited.
5. Go to step 2 if there are any nodes are unvisited.

Complexity Analysis of Dijkstra Algorithm:

- **Time complexity:** $O((V + E) \log V)$, where V is the number of vertices and E is the number of edges.
- **Auxiliary Space:** $O(V)$, where V is the number of vertices and E is the number of edges in the graph.